

---

## DSC 140B - Homework 03

Released: 21st May 2026, Due: 9th June 2026. 08:00 AM. Total Points: 100

---

Write your solutions to the following problems by typing them in  $\text{\LaTeX}$ . Unless otherwise noted by the problem's instructions, show your work or provide some justification for your answer. Homeworks are due via Gradescope at 08:00 AM.

### $\text{\LaTeX}$ learning materials:

1. HW Overleaf Template
2. Python +  $\text{\LaTeX}$ Tutorials

### Guidelines for Homework Related Questions:

1. As a general rule, we can help you understand the problems in the homework and explain the material from the corresponding lectures, but we cannot give you the entire solution.
2. Regarding debugging programming assignments: We ask you to do some debugging on your own first, including printing out intermediate values in your algorithms, trying a simpler version of the problem, etc.
3. We will not be pre-grading the homework, i.e. we won't confirm if the answer you have is correct.

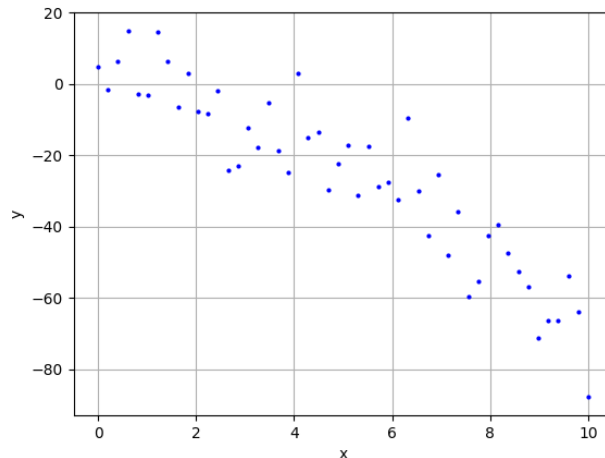
### Note:

1. The overleaf link is for you to copy it to your workspace, so it's “**view only**”, and cannot be edited.
2. Please review Homework Submission Instructions (Instructions Document) to understand what is acceptable format of solutions.
3. HW assignments are subject to a 20% late penalty per day. (Check out course syllabus for more details on late policy)
4. We will grade your submission on Gradescope and automatically apply slip day based on your submission time and slip day balance.

---

### Problem 1.

Consider fitting a prediction function to the data shown below:



The pattern in this data is not linear, but we can still fit it using a linear prediction function and an appropriate choice of basis functions.

The data for this problem can be found at:

[https://drive.google.com/file/d/1JJLH-EIo6-ovmQQtoy4pncQqjIDhYNYF/view?usp=share\\_link](https://drive.google.com/file/d/1JJLH-EIo6-ovmQQtoy4pncQqjIDhYNYF/view?usp=share_link)

- a) Assume that a good fit for the given data is a polynomial line. What do you think should be the degree ( $n$ ) of this polynomial?

*Hint:* You can do some basic analysis in python to figure this out.

(5 Points)

**Solution:** Although degree 5 gives the lowest training average square loss among the tested degrees, I would choose  $n = 2$  as a reasonable polynomial degree because the scatter plot mainly shows one smooth downward-curving trend. Higher-degree polynomials may reduce training loss slightly but can overfit the noise.

- b) For this and the next few subproblems, consider fitting a function of the form  $H_1(x; w) = wx^n$ . Note that this function has just one parameter to be learned:  $w$ . Here  $n$  is the degree of the polynomial that you thought was the best fit for the data - Part (a).

Write a Python function that takes in one argument,  $w$ , and computes the average square loss (that is, the mean squared error) of  $H_1$  on the data set with respect to  $w$ . Use your function to compute the average square loss of  $w = -0.6$ .

*Hint:* the correct output of your function when called on  $-0.6$  is a number between 100 and 200.

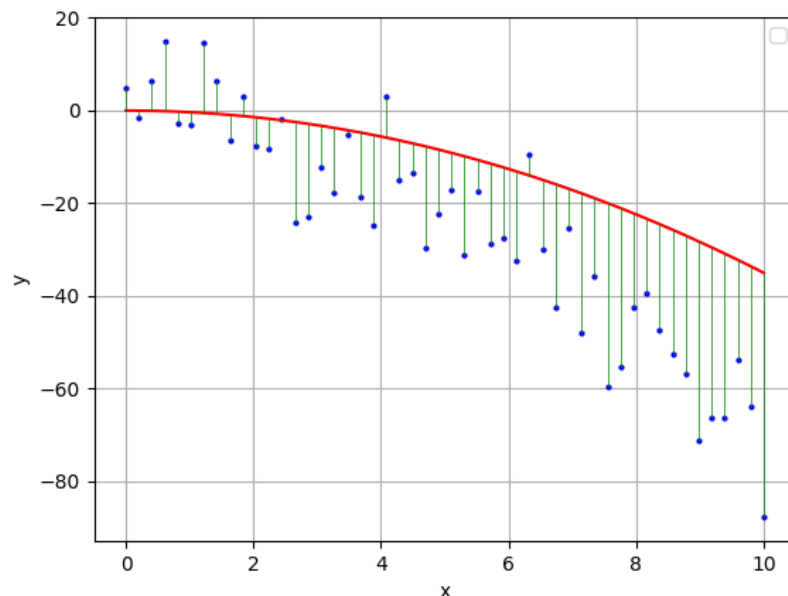
(5 Points)

**Solution:**

```
def avg_square_loss(w):  
    n = 2  
    y_pred = w * (x ** n)  
    loss = ((y - y_pred) ** 2).mean()  
    return loss
```

- c) The parameter  $w$  controls the value of  $wx^n$  when  $x$  is close to zero. In that sense, it is a scale parameter. Different values of  $w$  will change the shape of  $H_1$ , and will incur different amounts of error.

For example, the plot below shows  $H_1$  for a particular choice of  $w$  that happens to be too small in magnitude;  $H_1$  is not a good fit to the data. The green lines show the residuals. Your Python function from the last part computes the average squared length of these green lines.



One way to find the optimal value of  $w$  (that is, the value resulting in the smallest average squared residual) is to use a numerical optimizer, such as `scipy.optimize.minimize`.

Use `scipy.optimize.minimize` to find the value of  $w$  which minimizes the average square loss (mean squared error). Also, plot the fit line over the data and show the residuals.

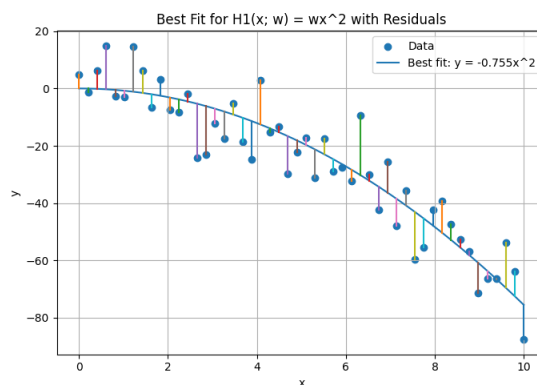
(5 Points)

**Solution:**

$$w^* \approx -0.7549847720351576$$

with  $n = 2$ . The minimum average square loss is

84.2820.



Best fit for  $H_1(x; w) = wx^2$  with residuals.

- d) Another way to find the optimal value of  $w$  is to map the data to “feature space” and to fit a straight line in that space. In this case, we have a single basis function  $\phi(x) = x^n$ . Therefore,  $H_1(x) = wx^n = w\phi(x)$ .

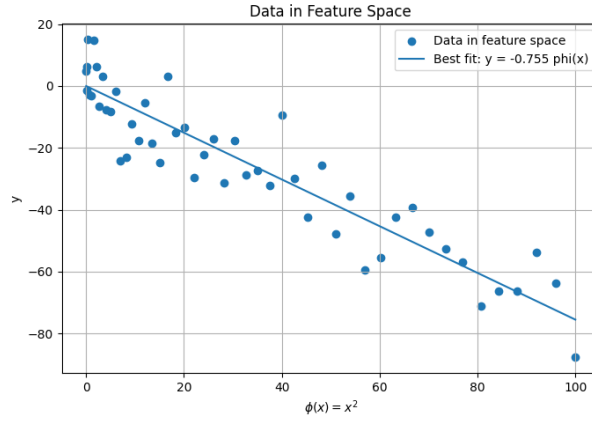
Plot the data in feature space; that is, create a “new” data set where each original point  $(x, y)$  becomes

$(\phi(x), y)$ .

*Hint:* you should see a linear trend.

(5 Points)

**Solution:**



- e) Perform linear least squares in feature space. That is, fit a straight line to the “new” data

$$(\phi(x^{(1)}), y_1), \dots, (\phi(x^{(n)}), y_n)$$

and report its slope,  $w$ .

Note: Perform least squares regression using `np.linalg.lstsq`. In this case, *do not* augment the data, since we are fitting a model  $w\phi(x)$  *without* a bias term.

(5 Points)

**Solution:**

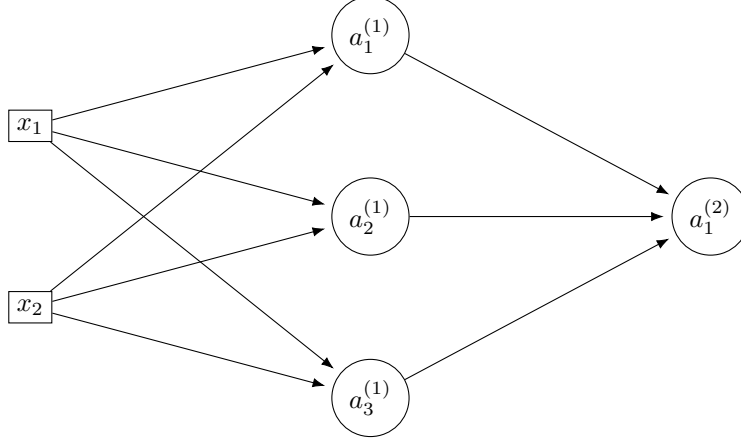
$$w \approx -0.7549847692052682.$$

## Problem 2.

In the lecture, it was said that a neural network with linear activation functions is a linear prediction function, meaning that its decision boundary will also be linear. If we wish to have a non-linear decision boundary, we must introduce non-linearities with, for instance, non-linear activation functions.

In this problem, we’ll see concretely that a neural network with linear activations, is again linear.

Consider the neural network shown below.



The inputs to the network is  $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ ,  $w_{ij}^{(l)} \in \mathbb{R}$  that connects neuron  $i$  in layer  $l-1$  to neuron  $j$  in layer  $l$ , and each  $b_i^{(l)} \in \mathbb{R}$  is the bias term for neuron  $i$  in layer  $l$ .  $a_i^{(l)}$  denotes the output of  $i^{th}$  neuron in layer  $l$ . Remember that when linear activations are used, the output of a neuron is simply the weighted sum of its inputs. We can represent the computations of this fully connected neural network using matrix notation as follows:

- **Input Layer:** The input vector to the network is:

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

- **Hidden Layer:** The input is passed to a hidden layer consisting of three neurons. The transformation is governed by the weight matrix  $W^{(1)}$  and bias vector  $b^{(1)}$ :

$$W^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ b_3^{(1)} \end{bmatrix}$$

The output of the hidden layer is:

$$a^{(1)} = \begin{bmatrix} a_1^{(1)} \\ a_2^{(1)} \\ a_3^{(1)} \end{bmatrix} = (W^{(1)})^\top x + b^{(1)}$$

- **Output Layer:** The hidden layer is followed by a single output neuron. The transformation is defined by:

$$W^{(2)} = \begin{bmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{bmatrix}, \quad b^{(2)} = [b_1^{(2)}]$$

The final output of the network is:

$$a^{(2)} = [a_1^{(2)}] = (W^{(2)})^\top a^{(1)} + b^{(2)}$$

- Write the output of the network,  $a_1^{(2)}$ , as an expression involving only the inputs  $x_1, x_2$  weights,  $w_{ij}^{(l)}$ , and bias  $b_i^{(l)}$ . Note that  $a_i^{(l)}$  should not appear in your expression.

**(5 Points)**

**Solution:**

$$a_1^{(1)} = w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + b_1^{(1)},$$

$$a_2^{(1)} = w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)},$$

$$a_3^{(1)} = w_{13}^{(1)} x_1 + w_{23}^{(1)} x_2 + b_3^{(1)}.$$

The output neuron is

$$a_1^{(2)} = w_{11}^{(2)} a_1^{(1)} + w_{21}^{(2)} a_2^{(1)} + w_{31}^{(2)} a_3^{(1)} + b_1^{(2)}.$$

Substituting the hidden-layer outputs gives

$$\begin{aligned} a_1^{(2)} &= w_{11}^{(2)} (w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + b_1^{(1)}) \\ &\quad + w_{21}^{(2)} (w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}) \\ &\quad + w_{31}^{(2)} (w_{13}^{(1)} x_1 + w_{23}^{(1)} x_2 + b_3^{(1)}) + b_1^{(2)}. \end{aligned}$$

b) Show that the output of the network can be written

$$a_1^{(2)} = w_0 + w_1 x_1 + w_2 x_2,$$

where  $w_0, w_1$ , and  $w_2$  are scalars that depend only on the weights in the original network. By showing this, you're proving that the network above is equivalent to a much simpler linear model.

(5 Points)

**Solution:** Rearrange the result from a):

$$a_1^{(2)} = w_0 + w_1 x_1 + w_2 x_2,$$

where

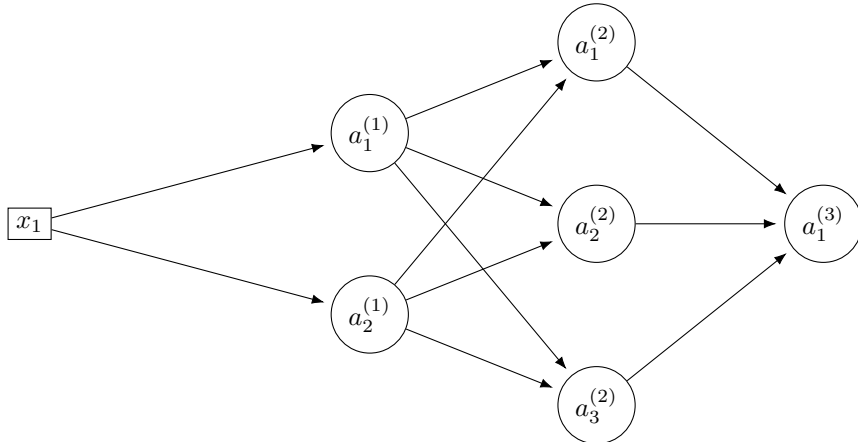
$$w_0 = w_{11}^{(2)} b_1^{(1)} + w_{21}^{(2)} b_2^{(1)} + w_{31}^{(2)} b_3^{(1)} + b_1^{(2)},$$

$$w_1 = w_{11}^{(2)} w_{11}^{(1)} + w_{21}^{(2)} w_{12}^{(1)} + w_{31}^{(2)} w_{13}^{(1)},$$

$$w_2 = w_{11}^{(2)} w_{21}^{(1)} + w_{21}^{(2)} w_{22}^{(1)} + w_{31}^{(2)} w_{23}^{(1)}.$$

### Problem 3.

Consider the neural network architecture shown below:



where  $a_i^{(l)} = \sigma \left( \sum_{j=1}^{n^{(l-1)}} w_{ji}^{(l)} a_j^{(l-1)} + b_i^{(l)} \right)$  (i.e  $a^{(l)} = \sigma((W^{(l)})^\top a^{(l-1)} + b^{(l)})$ ),  $a_1^{(0)} = x_1$ ,  $n^{(l)}$  is the number of neurons in layer  $l$ , and  $\sigma$  is the activation function. In all parts of this problem, assume that the network's parameters are:

$$W^{(1)} = \begin{bmatrix} -4 & 3 \end{bmatrix} \quad W^{(2)} = \begin{bmatrix} -3 & 2 & 1 \\ -4 & -2 & 5 \end{bmatrix} \quad W^{(3)} = \begin{bmatrix} -3 \\ 2 \\ 5 \end{bmatrix}$$

$$b^{(1)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \quad b^{(2)} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \quad b^{(3)} = [0]$$

Note that this network is a function  $H : \mathbb{R} \rightarrow \mathbb{R}$  such that  $H(x_1) = a_1^{(3)}$ , so we can easily plot it. In the parts below, you will plot the network for a range of inputs. Your plots may not necessarily be *interesting*, but they will give you a sense of how different choices of activation function affect the type of function the neural network computes.

*Suggestion:* create a Python function `network(x, activation)` which takes in two things: a number `x` and an `activation` function, and computes the output of the network on  $x$  using that activation function (that is, it computes  $H(x)$ ). You can then use that code for all parts of this problem.

- a) Assume that all activation functions are linear. Plot  $H(x)$  in the range  $x \in [-3, 3]$ . Show your code.

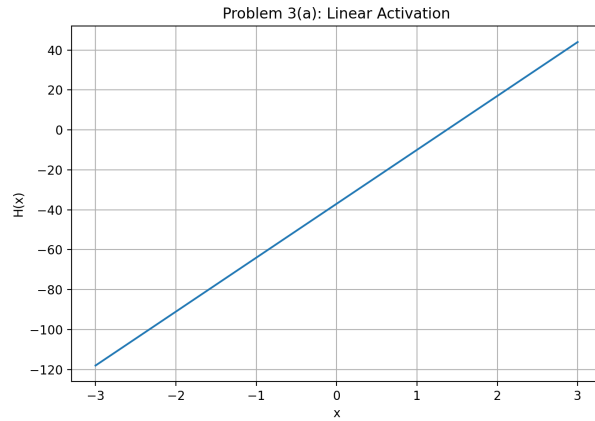
(5 Point)

**Solution:**

```
def network(x, activation):
    a0 = np.array([x])
    z1 = W1.T @ a0 + b1
    a1 = activation(z1)
    z2 = W2.T @ a1 + b2
    a2 = activation(z2)
    z3 = W3.T @ a2 + b3
    return z3[0]

def linear(z):
    return z

xs = np.linspace(-3, 3, 300)
ys_linear = np.array([network(x, linear) for x in xs])
plt.plot(xs, ys_linear)
plt.xlabel("x")
plt.ylabel("H(x)")
plt.title("Problem 3(a): Linear Activation")
plt.grid(True)
plt.tight_layout()
plt.show()
```



- b) Assume that all hidden nodes have **sigmoid** activation and that the output node has linear activation. Plot  $H(x)$  in the range  $x \in [-3, 3]$ . Show your code.

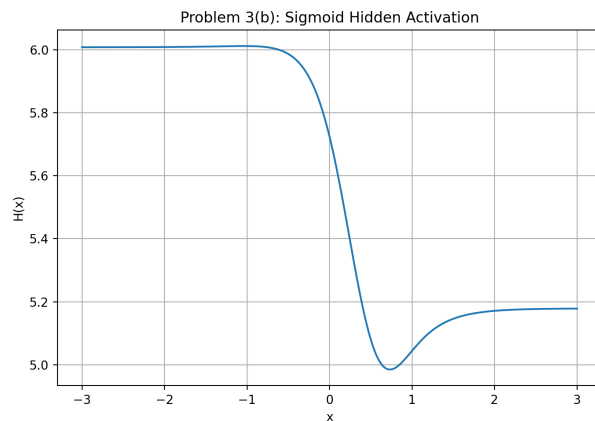
(5 Point)

**Solution:**

With the same network function as in 3(a):

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

xs = np.linspace(-3, 3, 300)
ys_sigmoid = np.array([network(x, sigmoid) for x in xs])
plt.plot(xs, ys_sigmoid)
plt.xlabel("x")
plt.ylabel("H(x)")
plt.title("Problem 3(b): Sigmoid Hidden Activation")
plt.grid(True)
plt.tight_layout()
plt.show()
```



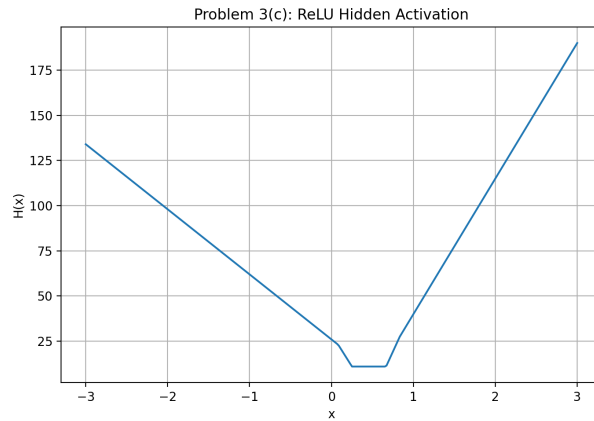
- c) Assume that all hidden nodes have **ReLU** activation and that the output node has linear activation. Plot  $H(x)$  in the range  $x \in [-3, 3]$ . Show your code.

(5 Point)

**Solution:**

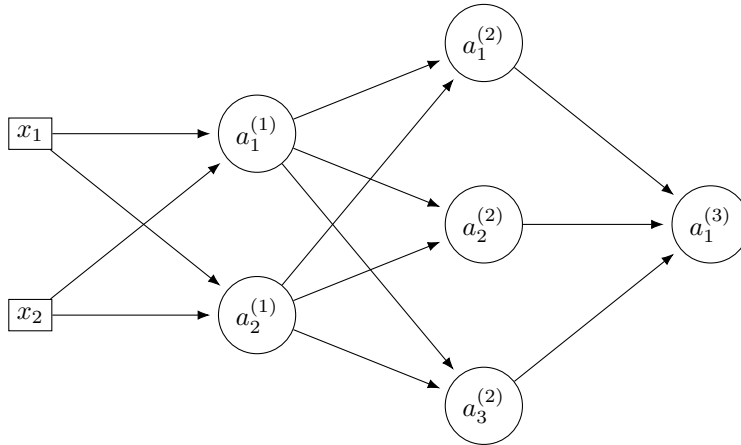
With the same network function as in 3(a):

```
def relu(z):  
    return np.maximum(0, z)  
  
xs = np.linspace(-3, 3, 300)  
ys_relu = np.array([network(x, relu) for x in xs])  
plt.plot(xs, ys_relu)  
plt.xlabel("x")  
plt.ylabel("H(x)")  
plt.title("Problem 3(c): ReLU Hidden Activation")  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```



**Problem 4.**

Consider the neural network  $H$  shown below.



where  $a_i^{(l)} = \sigma \left( \sum_{j=1}^{n^{(l-1)}} w_{ji}^{(l)} a_j^{(l-1)} \right)$  (i.e.  $a^{(l)} = \sigma((W^{(l)})^\top a^{(l-1)})$ ),  $n^{(l)}$  is the number of neurons in layer  $l$ ,  $a_1^{(0)} = x_1$ ,  $a_2^{(0)} = x_2$ ,  $H(x) = a_1^{(3)}$ , and  $\sigma$  is the activation function. Note that for simplicity, there are no

bias weights. Assume that the hidden layers use the ReLU as their activation function, and that the output layer uses the linear activation.

Suppose that  $x = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$  and that the weights of this network are:

$$W^{(1)} = \begin{bmatrix} 1 & 1 \\ -2 & 0 \end{bmatrix} \quad W^{(2)} = \begin{bmatrix} 3 & 1 & 3 \\ 2 & -2 & 0 \end{bmatrix} \quad W^{(3)} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$$

Carry out backpropagation in order to fill in the tables of partial derivatives below. Each entry should be a number. Show your work.

**Hint:** Few entries of the tables have been filled in for you – use these to check your work. **(15 Points)**

|                                            |   |                                            |   |  |  |
|--------------------------------------------|---|--------------------------------------------|---|--|--|
|                                            |   | $\frac{\partial H}{\partial W_{11}^{(2)}}$ | - |  |  |
|                                            |   | $\frac{\partial H}{\partial W_{12}^{(2)}}$ | - |  |  |
|                                            |   | $\frac{\partial H}{\partial W_{13}^{(2)}}$ | - |  |  |
|                                            |   | $\frac{\partial H}{\partial W_{21}^{(2)}}$ | 4 |  |  |
|                                            |   | $\frac{\partial H}{\partial W_{22}^{(2)}}$ | 0 |  |  |
|                                            |   | $\frac{\partial H}{\partial W_{23}^{(2)}}$ | - |  |  |
| $\frac{\partial H}{\partial W_{11}^{(1)}}$ | - |                                            |   |  |  |
| $\frac{\partial H}{\partial W_{12}^{(1)}}$ | - |                                            |   |  |  |
| $\frac{\partial H}{\partial W_{21}^{(1)}}$ | - |                                            |   |  |  |
| $\frac{\partial H}{\partial W_{22}^{(1)}}$ | 8 |                                            |   |  |  |

|                                            |   |
|--------------------------------------------|---|
| $\frac{\partial H}{\partial W_{11}^{(3)}}$ | 4 |
| $\frac{\partial H}{\partial W_{21}^{(3)}}$ | 0 |
| $\frac{\partial H}{\partial W_{31}^{(3)}}$ | - |

**Solution:**

$$x = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

$$z^{(1)} = (W^{(1)})^T x.$$

$$z_1^{(1)} = 1(2) + (-2)(2) = -2,$$

$$z_2^{(1)} = 1(2) + 0(2) = 2.$$

$$a^{(1)} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

$$z^{(2)} = (W^{(2)})^T a^{(1)}.$$

$$z_1^{(2)} = 3(0) + 2(2) = 4,$$

$$z_2^{(2)} = 1(0) + (-2)(2) = -4,$$

$$z_3^{(2)} = 3(0) + 0(2) = 0.$$

$$a^{(2)} = \begin{bmatrix} 4 \\ 0 \\ 0 \end{bmatrix}.$$

$$H = a_1^{(3)} = 2(4) + 1(0) + 2(0) = 8.$$

$$\frac{\partial H}{\partial W^{(3)}} = \begin{bmatrix} 4 \\ 0 \\ 0 \end{bmatrix}.$$

$$\delta^{(2)} = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix},$$

$$\frac{\partial H}{\partial W^{(2)}} = \begin{bmatrix} 0 & 0 & 0 \\ 4 & 0 & 0 \end{bmatrix}.$$

$$\delta^{(1)} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}.$$

$$\frac{\partial H}{\partial W^{(1)}} = \begin{bmatrix} 0 & 8 \\ 0 & 8 \end{bmatrix}.$$

$$\frac{\partial H}{\partial W^{(1)}} = \begin{bmatrix} 0 & 8 \\ 0 & 8 \end{bmatrix}, \quad \frac{\partial H}{\partial W^{(2)}} = \begin{bmatrix} 0 & 0 & 0 \\ 4 & 0 & 0 \end{bmatrix}, \quad \frac{\partial H}{\partial W^{(3)}} = \begin{bmatrix} 4 \\ 0 \\ 0 \end{bmatrix}.$$

**Problem 5.**

Consider a single-layer neural network used for binary classification (i.e., predicting 0 or 1). The network takes a two-dimensional input vector  $\vec{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$  and computes the output through the following steps:

1. **Linear combination:**

$$z = w_1 x_1 + w_2 x_2 + b$$

where  $\vec{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$  are the weights and  $b$  is the bias.

2. **Sigmoid Activation:**

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where  $\hat{y}$  is the predicted probability that the input belongs to class 1.

Derive an expression for  $\frac{\partial \hat{y}}{\partial w_1}$ ,  $\frac{\partial \hat{y}}{\partial w_2}$ , and  $\frac{\partial \hat{y}}{\partial b}$  in terms of  $\hat{y}$ ,  $x_1$  and  $x_2$ .

**(10 Point)**

**Solution:**

$$\frac{\partial \hat{y}}{\partial z} = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y}).$$

$$\frac{\partial \hat{y}}{\partial w_1} = \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w_1} = \hat{y}(1 - \hat{y})x_1.$$

$$\frac{\partial \hat{y}}{\partial w_2} = \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w_2} = \hat{y}(1 - \hat{y})x_2,$$

$$\frac{\partial \hat{y}}{\partial b} = \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial b} = \hat{y}(1 - \hat{y}).$$

### Problem 6.

In this assignment, you will implement and compare two different neural network architectures for image classification on the FashionMNIST dataset (Official Dataset link). The dataset contains 70,000 grayscale images of clothing items, with 60,000 images for training and 10,000 images for testing. Each image is  $28 \times 28$  pixels in size and belongs to one of 10 predefined classes. Your task is to implement each architecture using building block in PyTorch's `nn` class and evaluate their performance on the test set.

### Code Instructions

1. **Data Loading and Preprocessing:** Complete the code to load and preprocess the FashionMNIST dataset. Note that you only need to do this once and reuse dataset classes across models. **(5 Point)**
2. **Model 1 - Multi-Layer Perceptron (MLP):** **(5 Point)**
  - Implement an `MLP(nn.Module)` class in python.
  - The input to the model is a batch of flattened images of size **(batch\_size, 784)** and outputs are logits for each image with size **(batch\_size, 10)**. **Note:** Logits refer to the raw, unnormalized scores output by the model before applying any activation function like softmax. These logits are typically passed to a loss function `CrossEntropyLoss`, which internally applies softmax to convert them into probabilities during training.
  - The MLP should have three **Linear hidden layers** with sizes [512, 256, 128], with each hidden layers followed by **ReLU** non-linear activation function and output layer using linear activation function.

The network can be represented as:

Input(784)  $\rightarrow$  Linear(784  $\rightarrow$  512)  $\rightarrow$  ReLU  $\rightarrow$  Linear(512  $\rightarrow$  256)  $\rightarrow$  ReLU  $\rightarrow$  Linear(256  $\rightarrow$  128)  $\rightarrow$  ReLU  $\rightarrow$  Linear(128  $\rightarrow$  10)

3. **Model 2 - Convolutional Neural Network (CNN):** **(5 Point)**
    - Implement a `CNN(nn.Module)` class.
    - The input is a batch of grayscale images of shape **(batch\_size, 1, 28, 28)**. **Note:** The second dimension has size 1 because the images are grayscale (i.e., they have only one color channel, unlike RGB images which have three channels).
    - The first **Convolutional layer** must have:
      - `Input channels = 1`
      - `Output channels = 64`
      - `Kernel size = 3`
      - `Padding = 1`
      - `Stride = 1`
- followed by **ReLU** non-linear activation function.
- Note:** This is a *resolution-preserving* operation, meaning that the output feature map has the same spatial dimensions (height and width) as the input.
- This is followed by **MaxPooling** with `kernel size = 2`.

**Note:** This is a *downsampling* operation, which reduces the spatial dimensions (height and width) of the feature map by a factor of 2.

- The second convolutional layer must have:
  - **Input channels** = 64
  - **Output channels** = 64
  - **Kernel size** = 3
  - **Stride** = 1
  - **Padding** = 1

followed by ReLU non-linear activation function.

- This is followed by **MaxPooling** with **kernel size** = 2.
- After the feature extraction part, **flatten** the output feature maps and pass them through a `torch.nn.Linear` layer that outputs logits for each of the 10 classes.

The network can be represented as:

Input(1×28×28) → Convolution → ReLU → MaxPool → Convolution → ReLU → MaxPool → Flatten → Linear(\* → 10)

4. **Training Loop:** Implement the training loop with `CrossEntropyLoss` loss and `ADAM` optimizer with a batch size of 128 and train for 10 epochs. **(5 Point)**

Now evaluate each model's accuracy on the test set and answer the following questions: **(5 Point)**

1. Which model performed best on the test set? Provide the final accuracy scores for each.
2. Plot the training curve for each model and compare the results.

**Solution:** The CNN performed better on the FashionMNIST test set.

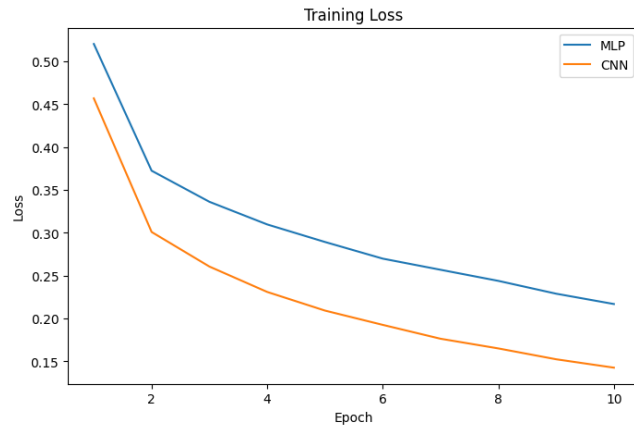
The final test results were:

MLP Test Loss = 0.323,   MLP Test Accuracy = 88.63%.

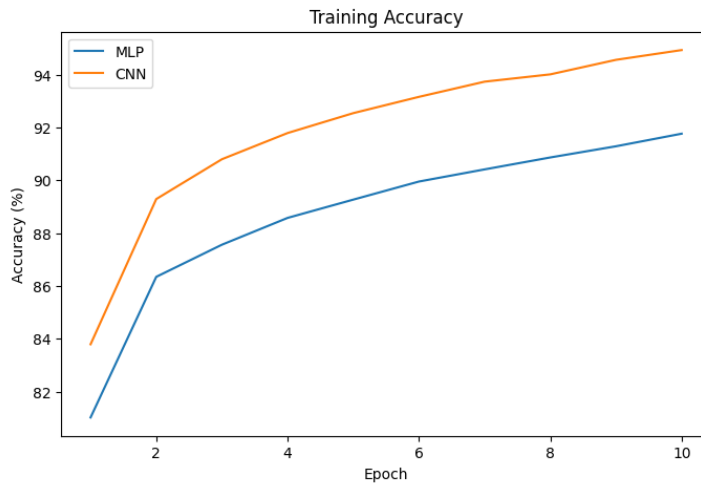
CNN Test Loss = 0.238,   CNN Test Accuracy = 91.76%.

Therefore, the CNN achieved a higher test accuracy and lower test loss than the MLP.

From the training curves, both models improved over the 10 epochs: the training loss decreased and the training accuracy increased. However, the CNN consistently had lower training loss and higher training accuracy than the MLP. This is expected because the CNN uses convolutional layers, which are better suited for image data because they preserve local spatial structure. The MLP flattens the image into a vector, so it loses some spatial information.



Training loss curves for the MLP and CNN.



Training accuracy curves for the MLP and CNN.

**Implementation instructions:** A code template is provided at: [Google Colab Notebook](#), with clearly marked TODO sections. Please use this template to complete all required tasks. For submission:

- Export your completed notebook as a PDF and merge it with the PDF containing any written responses to create a single document.
- Submit both the merged PDF and the notebook file (.ipynb) on Gradescope.